Cloud Deployment Guide

A case Study for ATMOS Space Cargo GmbH
by Pratibha Yadav

April 13, 2026

Abstract

This document describes the complete cloud deployment of the Prime Number generator Microservice on Microsoft Azure. It focuses solely on the deployment architecture and does not cover development-environment setup, microservice implementation details, or containerization fundamentals.

This document explains how the application operates as a containerized service on an Azure virtual machine (Host), with access restricted exclusively through a WireGuard VPN gateway. No API ports are exposed to the public internet; all communication is routed through an encrypted VPN tunnel to ensure strong security and full auditability. Each execution of the microservice writes an entry to a PostgreSQL log table, which remains fully isolated from public access.

The architecture and technology choices reflect practical considerations and professional experience. For example, the microservice was implemented in C# based on existing expertise. Azure was selected due to prior cloud-architecture experience, and WireGuard was chosen for its security, lightweight design, and zero licensing cost. These decisions balance technical suitability, operational simplicity, and economic efficiency.

Section 4 includes a summary table highlighting the key cloud design choices, while Section 4.1 (State-of the-art) expands on the best-practice architecture that would be recommended if cost were not a constraint.

Contents

1 Architecture **5**
1.1 Traffic Flow 6

2 Step-by-Step Deployment **7**
2.1 Install Azure CLI 7
2.2 Create Resource Group 8
2.3 Create Virtual Network 9
2.4 Create Network Security Group 10
2.5 Create Virtual Machine 11
2.6 Install Docker on VM 12
2.7 Upload Files & Deploy 13
2.8 Configure WireGuard 13
2.9 Verify Deployment 14
2.9.1 Verify PostgreSQL DB 15

3 User Access Management **16**
3.1 Grant Access & Revoke access to a User 16

4 Cloud Design Choices - Explanation **17**
4.1 State of the Art 18

5 Security Summary **23**

6 Quick Reference - Commands **24**

7 Tools Used **25**

List of Figures

1 Architecture Design 5
2 Traffic Flow at a glance 6
3 Windows Terminal to manage Azure Resources 7
4 Resource Group 8
5 Virtual Network 9
6 Network Security Group 10
7 Virtual Machine 11
8 Docker Installation on VM 12
9 Push Image to Docker Hub 12
10 Upload Files & Deploy 13
11 Configure Wireguard 13
12 Successful API response on VPN 14
13 ERR_CONNECTION_REFUSED without VPN 14
14 Dbeaver PostgreSQL DB Snapshot 15

List of Tables

1 *Cloud Design Choices explanation* 17
2 *State of the art: Compute & orchestration* 18
3 *State of the art: Networking & access* 19
4 *State of the art: Database & storage* 20
5 *State of the art: Security & secrets* 21
6 State of the art: Observability & reliability 22
7 *Quick Security Summary* 23
8 *Command Reference* 24
9 *Tools used* 25

1 Architecture

A single Azure Virtual Machine hosts all three Docker containers inside an isolated Docker bridge network.The WireGuard container acts as the sole entry point, functioning as a VPN[1] gateway that bridges encrypted external traffic into the internal Docker network.

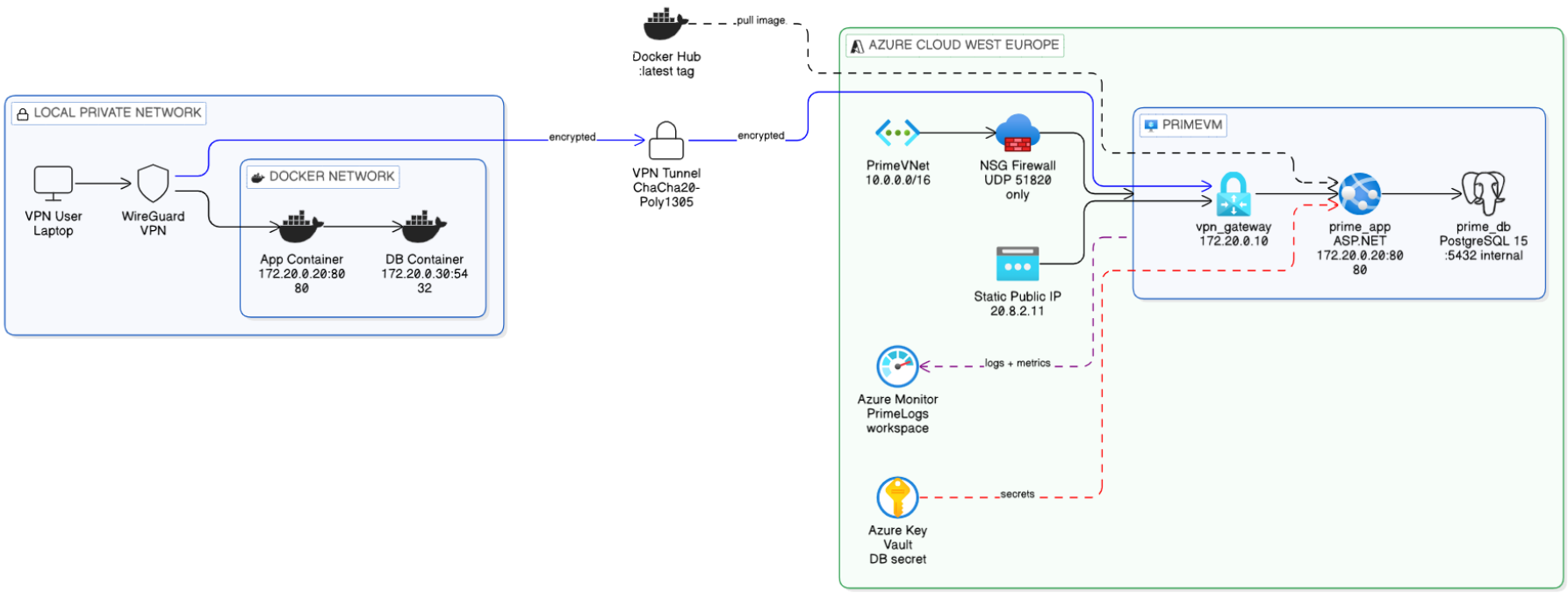

Figure 1: Architecture Design

[1] Advanced VPN and security options are covered in Section 4.1

1.1 Traffic Flow

The complete request lifecycle from a VPN-connected client to the database.

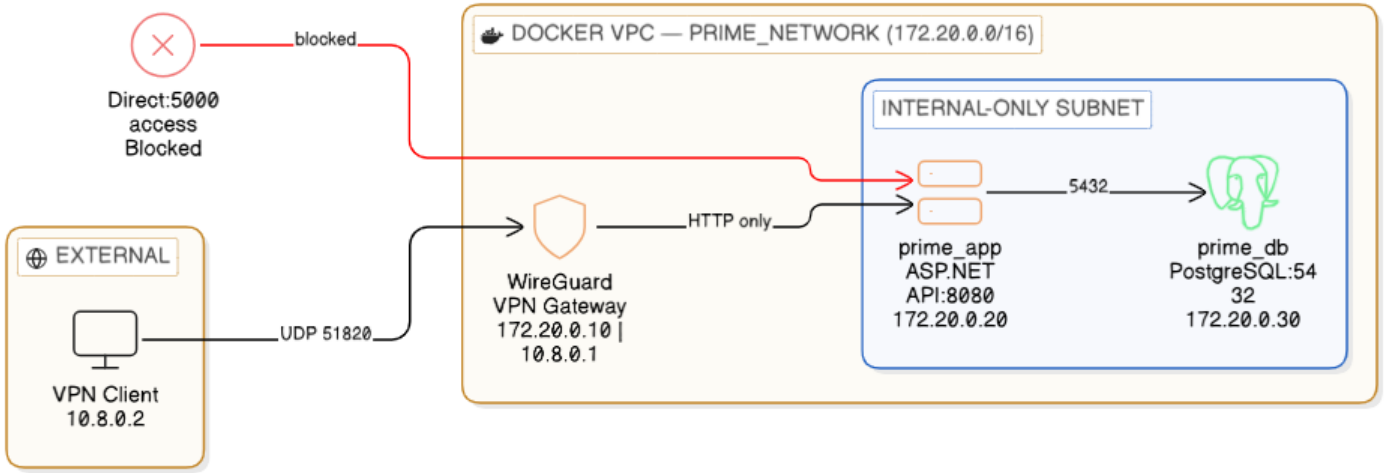

Figure 2: Traffic Flow at a glance

1. The client first activates WireGuard to create a secure VPN tunnel.
2. The client's browser sends a request to the URL or API.
3. Windows routes all traffic for the 172.20.x.x network through the WireGuard tunnel using the AllowedIPs rule. The traffic is fully encrypted over UDP.
4. Azure Network Security Group allows the UDP packet, and it reaches the virtual machine.
5. The WireGuard container on the VM decrypts the packet and forwards it to the Prime Number Microservice.
6. The microservice processes the request.
7. The result is saved in PostgreSQL, and the response is sent back through the same encrypted VPN path.

In the next section, we walk through each deployment step and how everything was created.

2 Step-by-Step Deployment

2.1 Install Azure CLI

This is used to Create and manage Azure resources from terminal

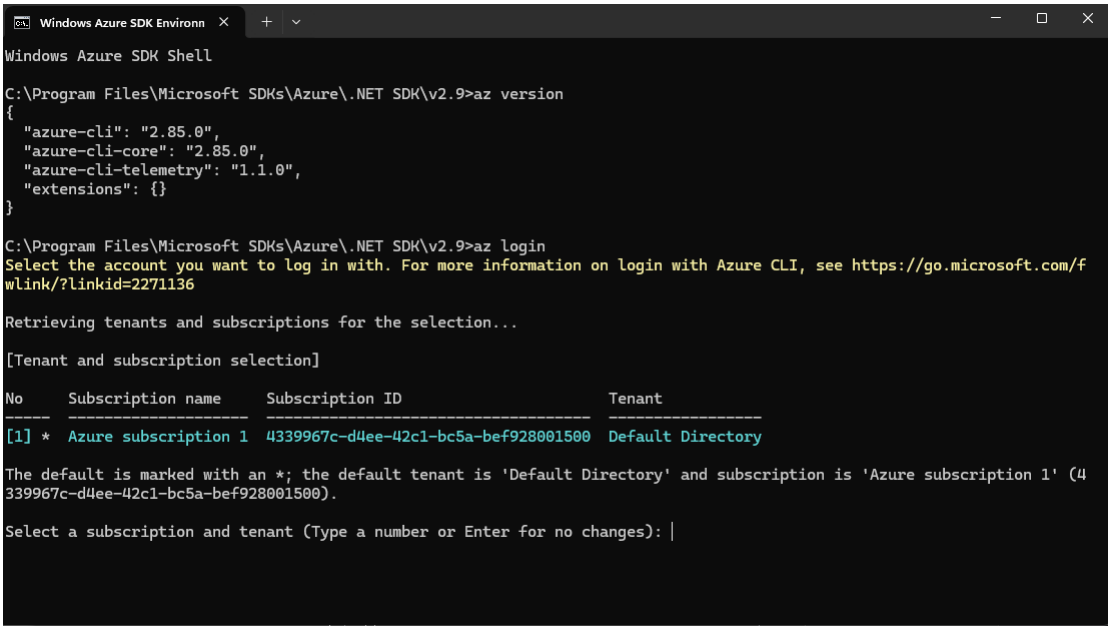

Figure 3: Windows Terminal to manage Azure Resources

2.2 Create Resource Group

A Resource Group is a logical container for all Azure resources belonging to this project.

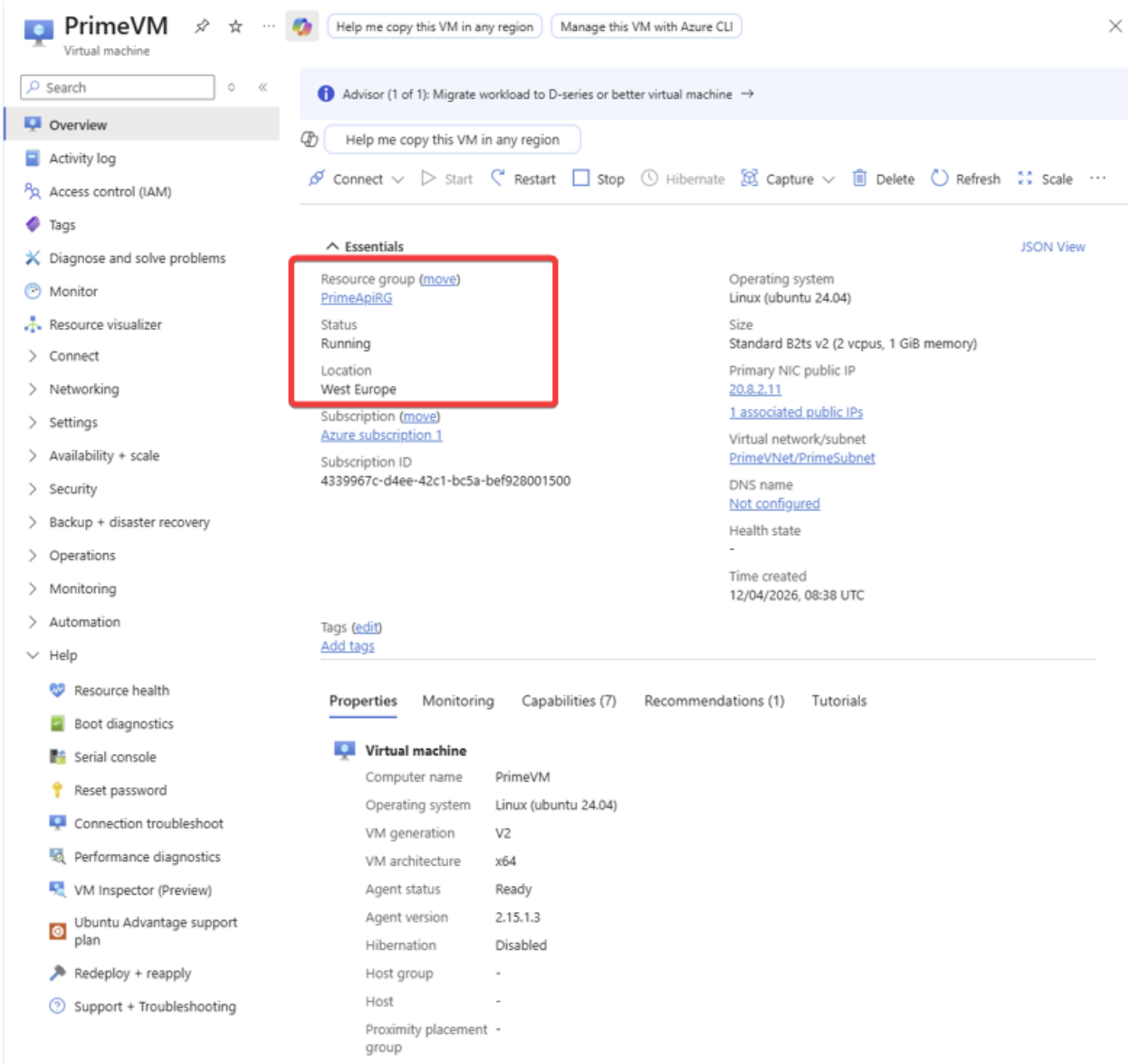

Figure 4: Resource Group

2.3 Create Virtual Network

The Azure VNet isolates all resources. It is the cloud equivalent of the local Docker bridge network. Nothing inside it is reachable from the internet unless you explicitly open a port.

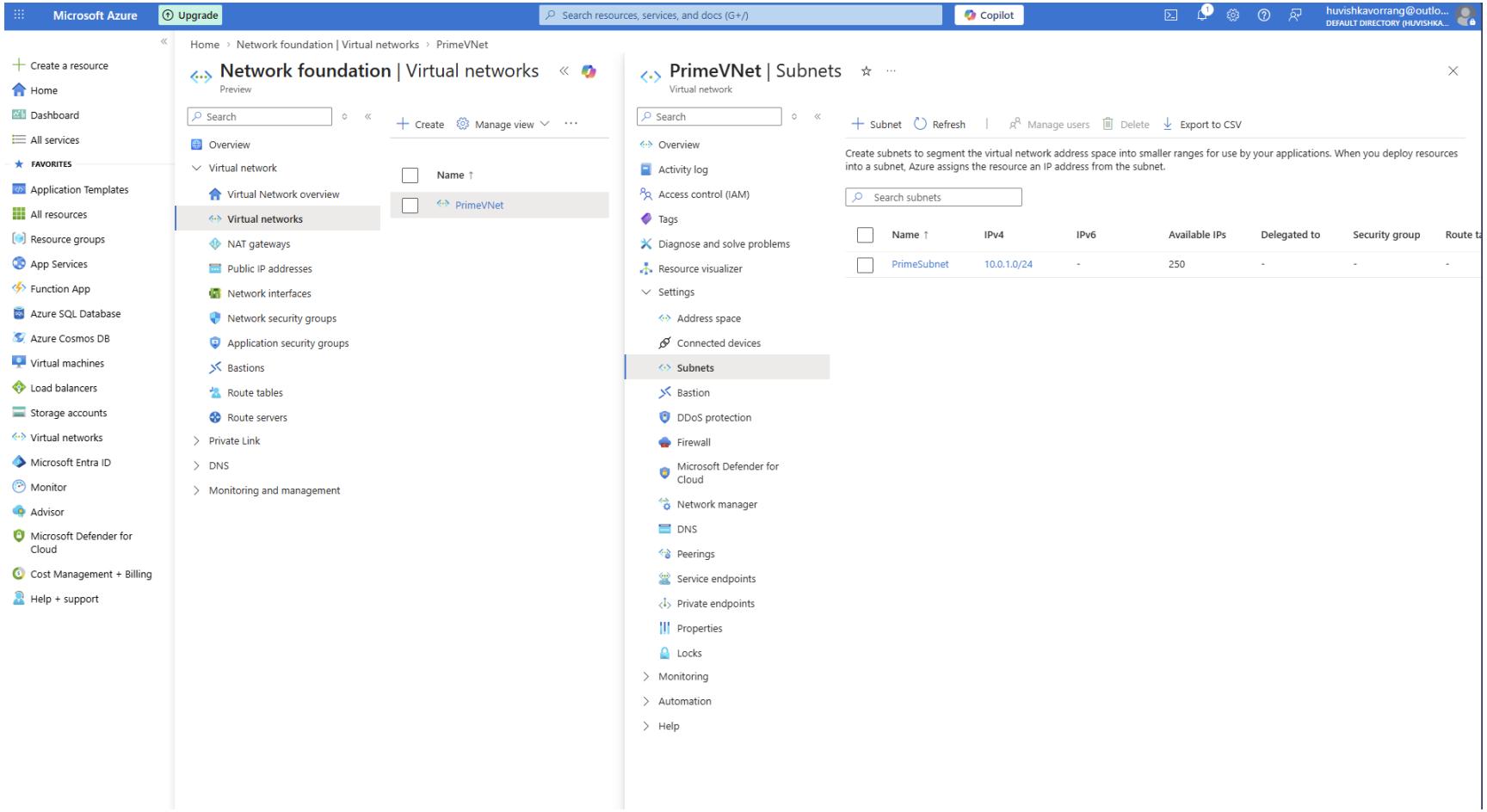

Figure 5: Virtual Network

2.4 Create Network Security Group

The NSG acts as a cloud firewall. This will allow only two types of inbound traffic: SSH[2] for management, and WireGuard VPN. All other ports including HTTP and PostgreSQL are blocked from the internet.

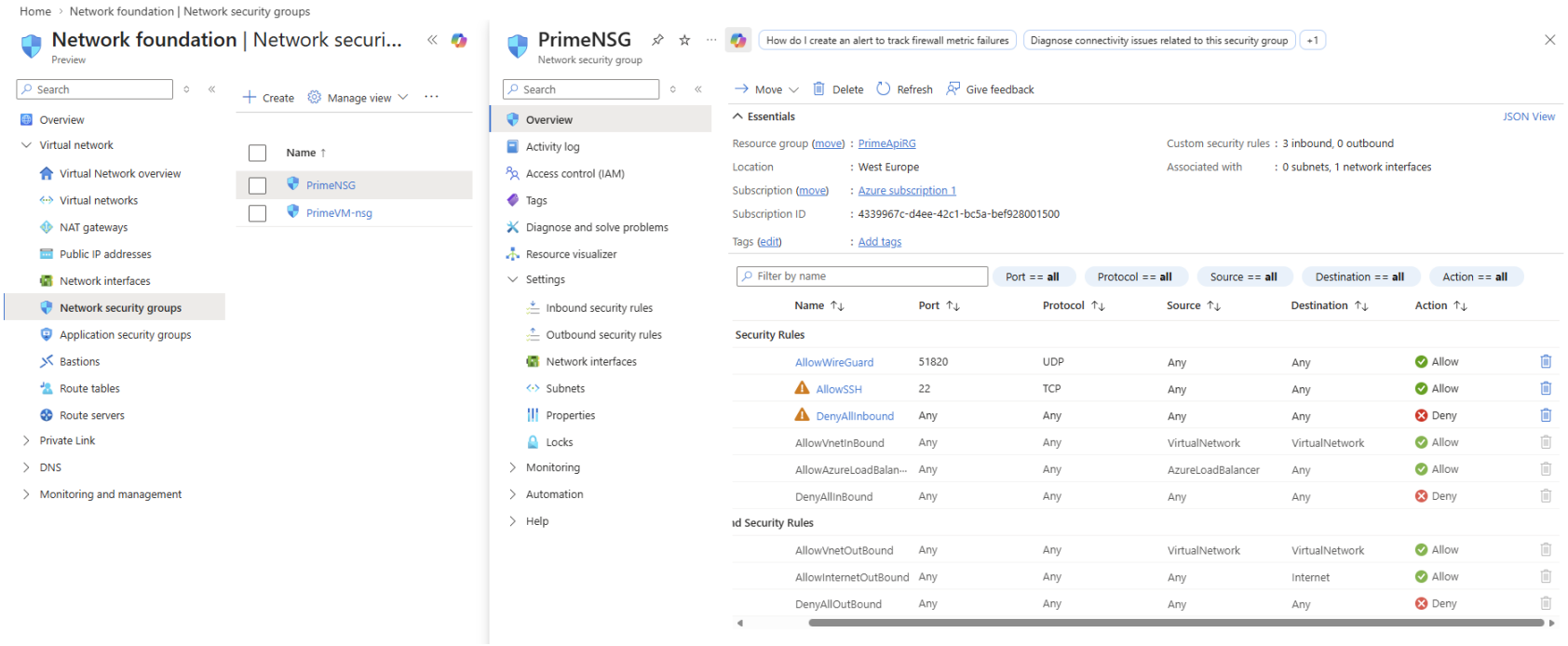

Figure 6: Network Security Group

[2]SSH is enabled only for the initial setup. After the VPN is connected, the Azure SSH rule for port 22 is removed

2.5 Create Virtual Machine

A virtual machine is created in Azure, as shown in the figure below.

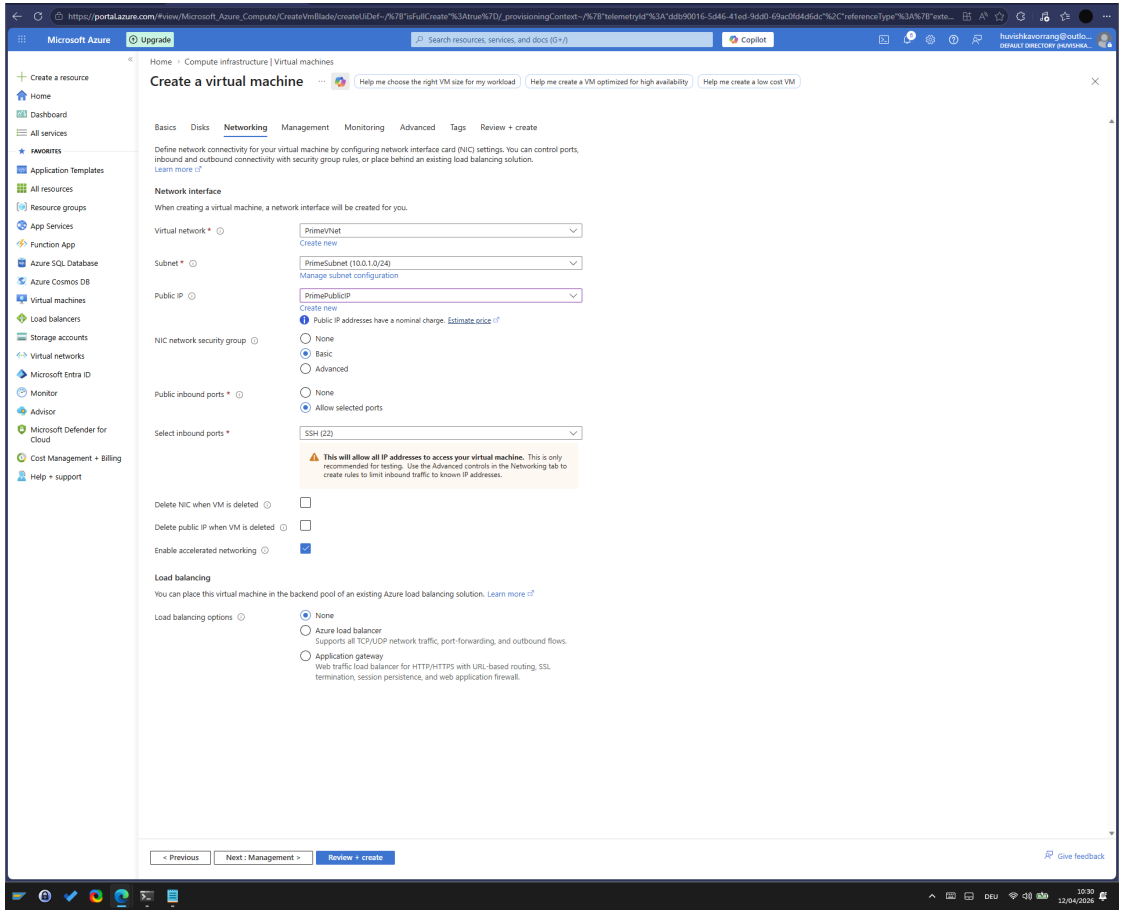

Figure 7: Virtual Machine

2.6 Install Docker on VM

SSH into the VM and install Docker:

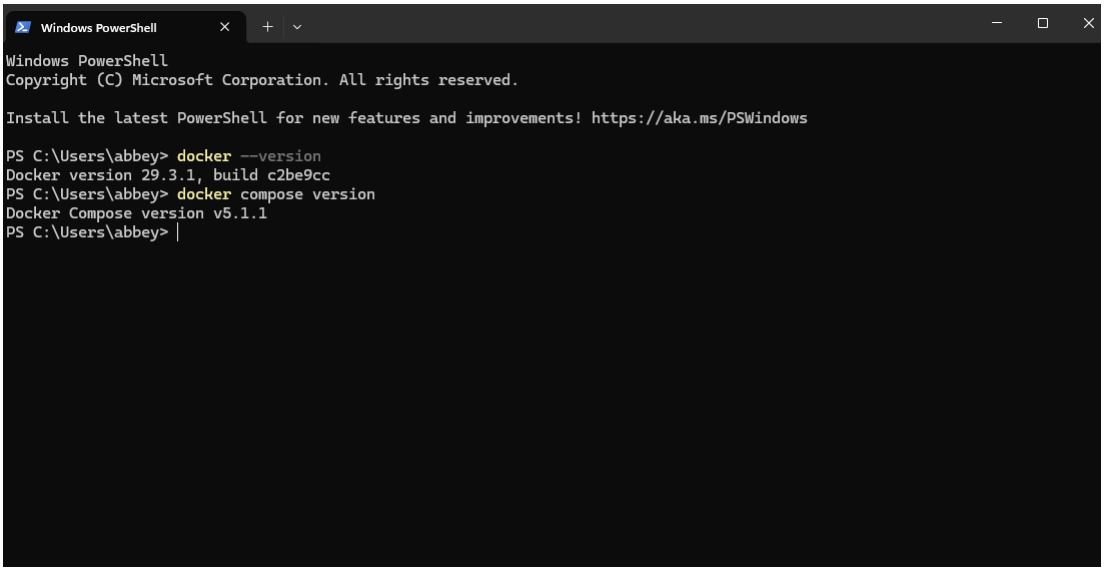

Figure 8: Docker Installation on VM

Building on the VM is slow (1 GB RAM). To make processing faster I build locally and push images to Docker Hub since the VM pulls the pre-built image in seconds

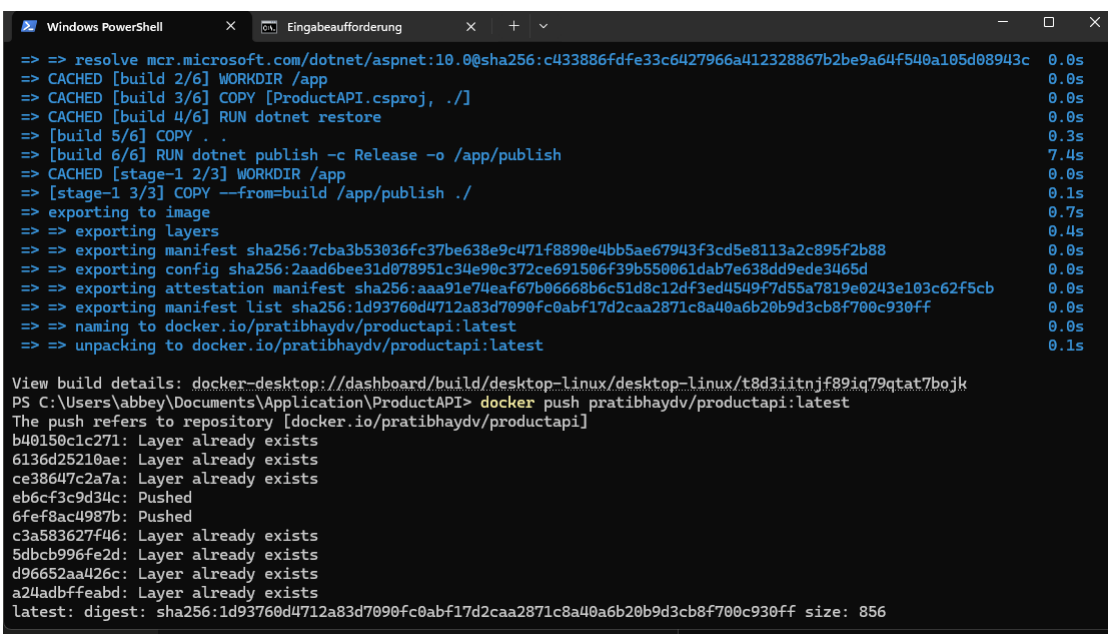

Figure 9: Push Image to Docker Hub

2.7 Upload Files & Deploy

In this step we Upload project files (PowerShell — local machine) using command scp -i ~/.ssh/primevm_key -r. Then we update docker-compose.yml on VM to use Docker Hub image instead of building locally using the command # On VM (SSH session) nano ~/ProductAPI/docker-compose.yml. Finally Change the app service configuration app: image: <Filename> # pull from Docker Hub # build: . # removed build happens locally, then Start all containers cd ~/ProductAPI docker compose up -d

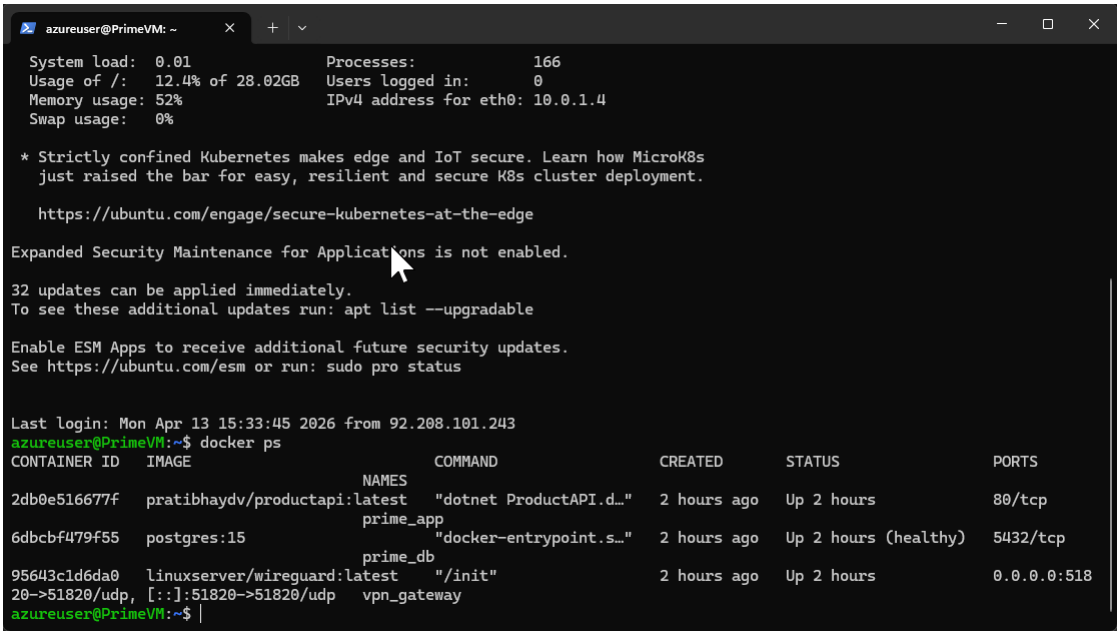

Figure 10: Upload Files & Deploy

2.8 Configure WireGuard

This file configures WireGuard as a SERVER. It has no Endpoint, it listens for incoming connections.

Figure 11: Configure Wireguard

2.9 Verify Deployment

At this stage we verfy the deployment with and without the Wireguard VPN

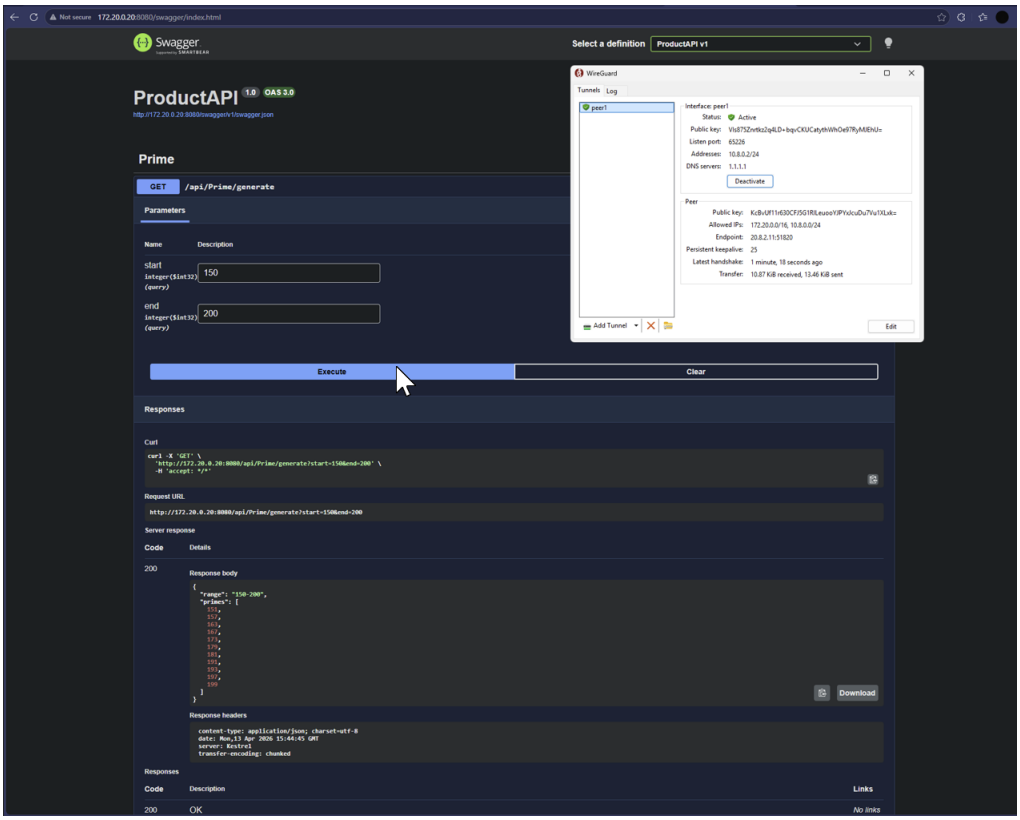

Figure 12: Successful API response on VPN

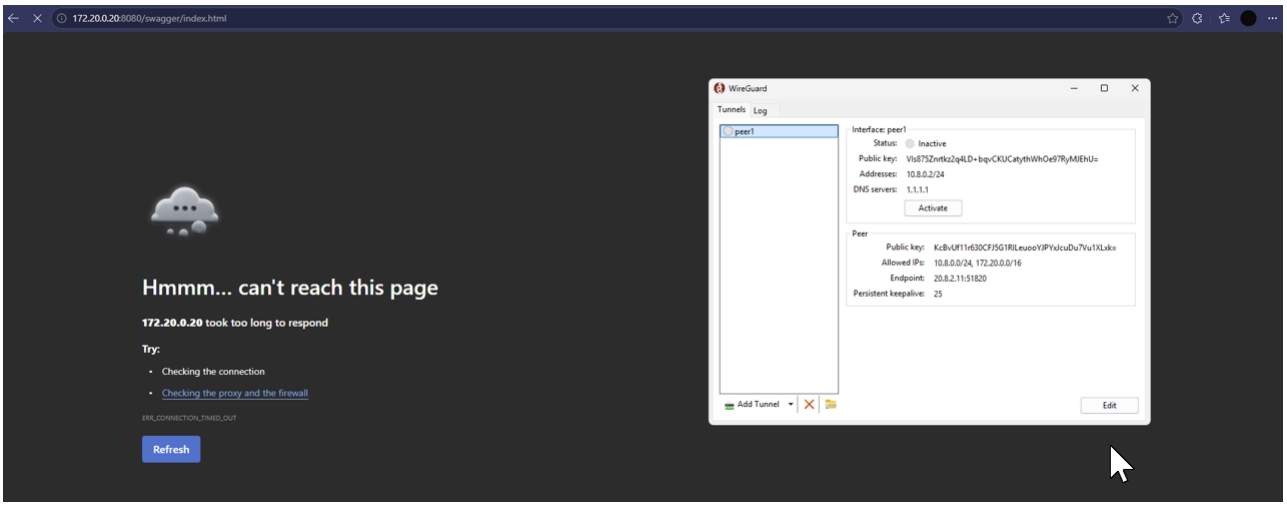

Figure 13: ERR_CONNECTION_REFUSED without VPN

2.9.1 Verify PostgreSQL DB

At this stage we can check all the queries that were prompted to the Prime number generator Microservice and are saved into the Database, the following screenshot demonstrates the active database

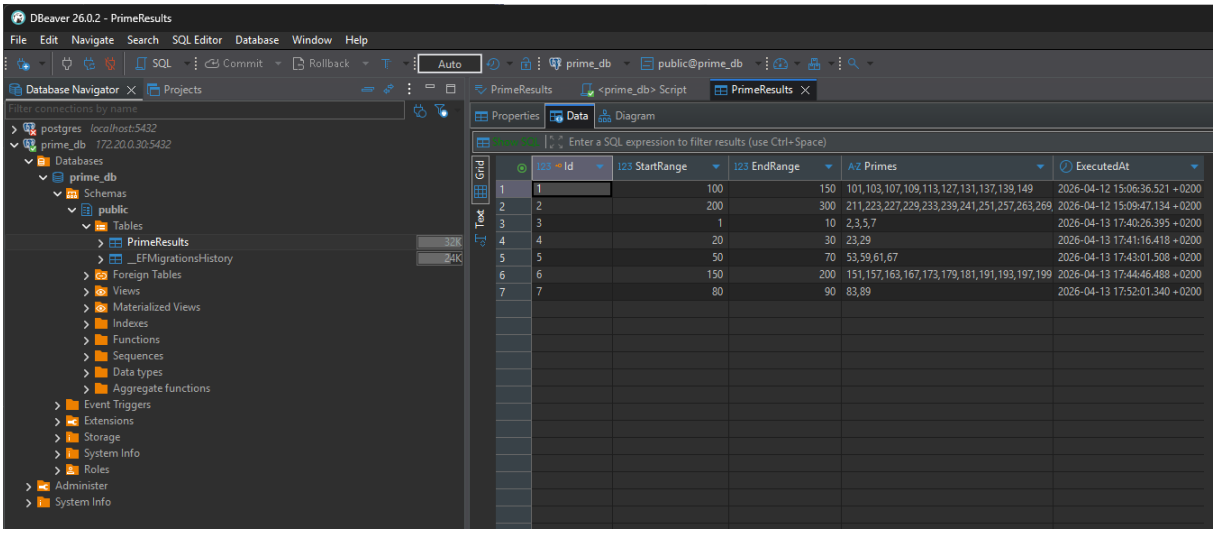

Figure 14: Dbeaver PostgreSQL DB Snapshot

3 User Access Management

In this case study, I was the only user, so there was no need for a full User Access Management setup. However, in an enterprise environment this becomes an essential requirement. Each user must receive their own unique WireGuard configuration file to access the API. Access is granted by adding the user's public key to the wg0.conf file, and revoked by removing it.

3.1 Grant Access & Revoke access to a User

To onboard a new user, the following steps are required:

1. Generate a WireGuard keypair for the user on the VM.
2. Add a new [Peer] block to wireguard/wg0.conf, assigning the next available IP address.
3. Restart the WireGuard service to apply the updated configuration.
4. Create and securely share the user's client configuration file.
5. Revoking Access To remove a user's access:
6. Delete the user's [Peer] block from wg0.conf.
7. Restart WireGuard (docker compose restart wireguard). Access is revoked immediately after the restart.

Note To monitor active connection we can make use of command docker exec vpn_gateway wg show inside the VM docker container and get information such as latest handshake Data transfer and allowed IP's (for details see figure 11)

Note:
To monitor active VPN connections, you can run the following command inside the WireGuard container on the VM:
docker exec vpn_gateway wg show
This displays useful information such as the latest handshake time, data transferred, and the list of allowed IPs. (See Figure 11 for details.).

4 Cloud Design Choices - Explanation

Table 1: *Cloud Design Choices explanation*

Tools	Purpose
Why a VM instead of Azure App Service?	Azure App Service cannot be used because it hides the underlying Docker and network configuration, so you cannot control the VPN-specific networking required for Task 2. A virtual machine is required because it gives full control: you install Docker yourself and run docker-compose.yml exactly as the assignment demands.
Why WireGuard over Azure VPN Gateway?	Azure VPN Gateway was not used because it is expensive and takes a long time to provision. WireGuard was chosen instead because it is free, starts quickly, and was already configured in Task 2. It is also a widely used, production-grade VPN.
Single VM with Docker Compose	It is a simple and low-cost setup. Docker Hub pulls can be slower, rate-limited, and public, while the VM can run WireGuard to provide secure VPN access.

4.1 State of the Art

Now that I've explained my design choices and why I made them, the next question is: what would the best-practice or state-of-the-art architecture look like if there were no monetary limitation?

Table 2: *State of the art: Compute & orchestration*

Category	What was used	State of the art
VM / server	Azure B2ts_v2(1 GB RAM · 2 vCPU)	**Azure Dedicated Host** Physical server reserved solely for you. No noisy neighbours. FIPS 140-2 hardware. Full hardware isolation — no other tenants on same physical machine. Compliance grade.
Orchestration	Docker Compose Single VM, manual restarts	**Azure Kubernetes Service (AKS)** Auto-scaling, self-healing pods, rolling deployments, zero-downtime updates If prime_app crashes it restarts in seconds automatically. Scale to 1000 replicas under load.
Container registry	Docker Hub (public) Public repo — anyone can pull	**Azure Container Registry Premium** Private registry with geo-replication, image signing, vulnerability scanning Images are private, signed, scanned for CVEs before deployment. Pulled from nearest region.

Table 3: *State of the art: Networking & access*

Category	What was used	State of the art
VPN	WireGuard (self-hosted) Manual key management, no MFA	**Azure VPN Gateway + Entra ID P2** MFA, conditional access, certificate-based auth, per-user policies, audit logs Every user needs MFA to connect. Auto-revoke on employee offboarding. Full audit trail.
Load balancer	None — single container Single point of failure	**Azure Application Gateway WAF v2** Web Application Firewall, SSL termination, DDoS protection, URL routing Blocks OWASP Top 10 attacks (SQL injection, XSS) before they reach your API.
DDoS protection	None Vulnerable to volumetric attacks	**Azure DDoS Protection Standard** Always-on traffic monitoring, automatic mitigation, $3000 cost credit guarantee Absorbs terabit-scale attacks. SLA-backed. Includes attack analytics and reports.

Table 4: *State of the art: Database & storage*

Category	What was used	State of the art
Database	PostgreSQL in Docker No backups, single instance, data lost if volume deleted	**Azure PostgreSQL Flexible Server Business Critical** 99.99% SLA, zone-redundant HA, point-in-time restore up to 35 days, read replicas If entire Azure datacenter goes down your DB fails over to another zone in under 120 seconds.
Backups	None Data lost on docker compose down -v	**Azure Backup + geo-redundant storage** Automated daily backups replicated to paired region 400 miles away Even if entire West Europe region burns down, restore from North Europe backup.

Table 5: *State of the art: Security & secrets*

Category	What was used	State of the art
Secret management	Azure Key Vault (free tier) Software-protected keys	**Azure Key Vault Premium (HSM)** Hardware Security Module — keys never leave physical HSM chip, FIPS 140-2 Level 3 Even Azure engineers cannot extract your private keys. Physically impossible.
Threat detection	Azure Defender basic (free) Basic alerts only	**Microsoft Defender for Cloud (all plans)** AI-powered threat intel, just-in-time VM access, adaptive network hardening, attack path analysis Detects when someone is probing your VM before they even find an open port.
Identity & access	WireGuard keypairs Manual key distribution	**Microsoft Entra ID P2 + PIM** Privileged Identity Management — just-in-time admin access, MFA everywhere, zero standing privileges Nobody has permanent admin access. Access granted for 1 hour, then auto-expires.

Table 6: State of the art: Observability & reliability

Category	What was used	State of the art
Monitoring	Azure Monitor (free tier) Basic logs, 5 GB free	**Azure Monitor + Application Insights + Sentinel** Full distributed tracing, SIEM, AI anomaly detection, custom dashboards, SLA alerting Know your API is slow before your users do. Security incidents auto-investigated
CI/CD pipeline	Manual — build & push locally Human runs docker push every time	**Azure DevOps + GitHub** Advanced Security Automated build, SAST/DAST scanning, secret scanning, signed releases, blue-green deployments Code with a SQL injection never reaches production — caught at PR stage automatically.
Uptime / SLA	~95% (single VM) VM reboot = downtime	**99.99% with AKS + zone redundancy** Multi-zone, multi-region, active-active failover 52 minutes of allowed downtime per year. Azure pays you if they miss it.

5 Security Summary

Table 7: *Quick Security Summary*

Security Control	Implementation
Network perimeter	Azure NSG only UDP 51820 allowed inbound
API access control	WireGuard VPN cryptographic authentication per user
Database isolation	PostgreSQL has no public port internal Docker network only
Per-user access	Per-user accessEach user has unique keypair, granular revocation
No shared passwords	WireGuard uses public-key cryptography, no password to leak
IP forwarding	Enabled only on NIC, required for VPN routing

6 Quick Reference - Commands

Table 8: *Command Reference*

Command	Where to run	Function
az vm start ...	Local PowerShell	Start the Azure VM
az vm deallocate ...	Local PowerShell	Stop + deallocate VM
ssh -i key azureuser@<Azure VM Public Key>	Local PowerShell	Connect to VM
docker compose up -d	SSH session (VM)	Start all containers
docker compose down	SSH session (VM)	Stop all containers
docker ps	SSH session (VM)	Check container status
docker logs prime_app	SSH session (VM)	Check app logs
docker exec vpn_gateway wg show	SSH session (VM)	Check VPN connections
docker compose restart wireguard	SSH session (VM)	Restart VPN gateway

7 Tools Used

Table 9: *Tools used*

Tools	Purpose
.NET 8 SDK	The SDK contains the C# compiler and the dotnet CLI.
Visual Studio Code	A free, lightweight code editor
Docker Desktop	To build and run containers on personal Machine
Docker Compose	Container orchestration
LaTex	For Documenting
Azure Cloud	Cloud Deployment
Dbeaver	Open-source database management tool and SQL client
Git/hub	Version Control & Task Submission